

METHODOLOGY DOCUMENTATION — APPROACH V1

Kaggle CMI Sensor Data → MediaPipe Feature Transformation

Technical Report: What Was Attempted, Results Achieved, and Why It Fails as a Production Solution

PROJECT

ENEB453 — BFRB Detection Web App

AUTHOR

Nishanth Tiglao

DATE

April 13, 2026

STATUS

Superseded — Documented for Reference

1. Executive Summary

This report documents the first technical approach attempted for real-time BFRB (Body-Focused Repetitive Behavior) detection: training a machine learning model on the **Kaggle Child Mind Institute (CMI) Detect Behavior with Sensor Data** dataset, then applying that model to live **MediaPipe** webcam data.

The approach achieved **77% accuracy and AUC-ROC 0.837** on the Kaggle test set, but produced **excessive false positives** in real webcam deployment due to a fundamental *domain gap* between the two data sources. This report explains exactly what was built, the results obtained, and the technical reasons this approach is insufficient for a production system.

KEY FINDING

The core problem is that we trained a classifier on physical wrist-sensor measurements (millimeters, degrees Celsius, m/s^2) and tried to apply it to image-coordinate distances from

a camera. Even after normalization, the decision boundaries learned from sensor data do not transfer cleanly to landmark distances, causing the model to fire on normal hand movements.

2. The Kaggle CMI Dataset

2.1 Overview

The **CMI Detect Behavior with Sensor Data** competition dataset was collected using a custom wrist-worn device called *Helios*. Subjects wore the bracelet while performing labeled BFRB gestures (e.g., nail biting, hair pulling, face touching) and control activities (e.g., texting, writing, waving).

Property	Value
Total rows (timesteps)	574,945
Total gesture sequences	8,151
BFRB sequences	5,272 (64.7%)
Non-BFRB sequences	2,879 (35.3%)
Total columns per row	341
Sensor sampling rate	~10 Hz (inferred)

2.2 Sensor Columns

Column(s)	Sensor Type	Units	Description
<code>acc_x, acc_y, acc_z</code>	Accelerometer (IMU)	m/s ²	3-axis wrist acceleration
<code>rot_w, rot_x, rot_y, rot_z</code>	Quaternion (IMU)	dimensionless	Wrist 3D orientation
<code>thm_1 .. thm_5</code>	Thermal (5 sensors)	°C	Skin contact warmth detection
<code>tof_1_v0 .. tof_5_v63</code>	Time-of-Flight (5 sensors, 8×8 grid)	mm	Spatial depth map around wrist (320 values total)

NOTE ON TOF SENSORS

Each ToF sensor produces an 8×8 pixel depth grid (64 values), making a spatial proximity map around the wrist. A value of -1 means out of range. These are particularly rich: when a finger approaches the face, the grid shows a cluster of low-distance readings (~50-100mm) rather than a single number.

2.3 Top Gestures in Dataset

Gesture	Category	Sequences
Forehead — pull hairline	BFRB	640
Neck — pinch skin	BFRB	640
Text on phone	No-BFRB	640
Neck — scratch	No-BFRB	640
Forehead — scratch	BFRB	640
Eyelash — pull hair	BFRB	640
Above ear — pull hair	BFRB	638
Eyebrow — pull hair	BFRB	638
Cheek — pinch skin	BFRB	637
Wave hello	No-BFRB	478

3. Feature Transformation Approach

3.1 Core Concept

The central hypothesis was that both the Kaggle wrist sensor and MediaPipe landmarks measure the same underlying physical reality — a hand performing a gesture — through different sensing modalities. If we could map both into a common semantic feature space, a model trained on Kaggle data could classify MediaPipe data.

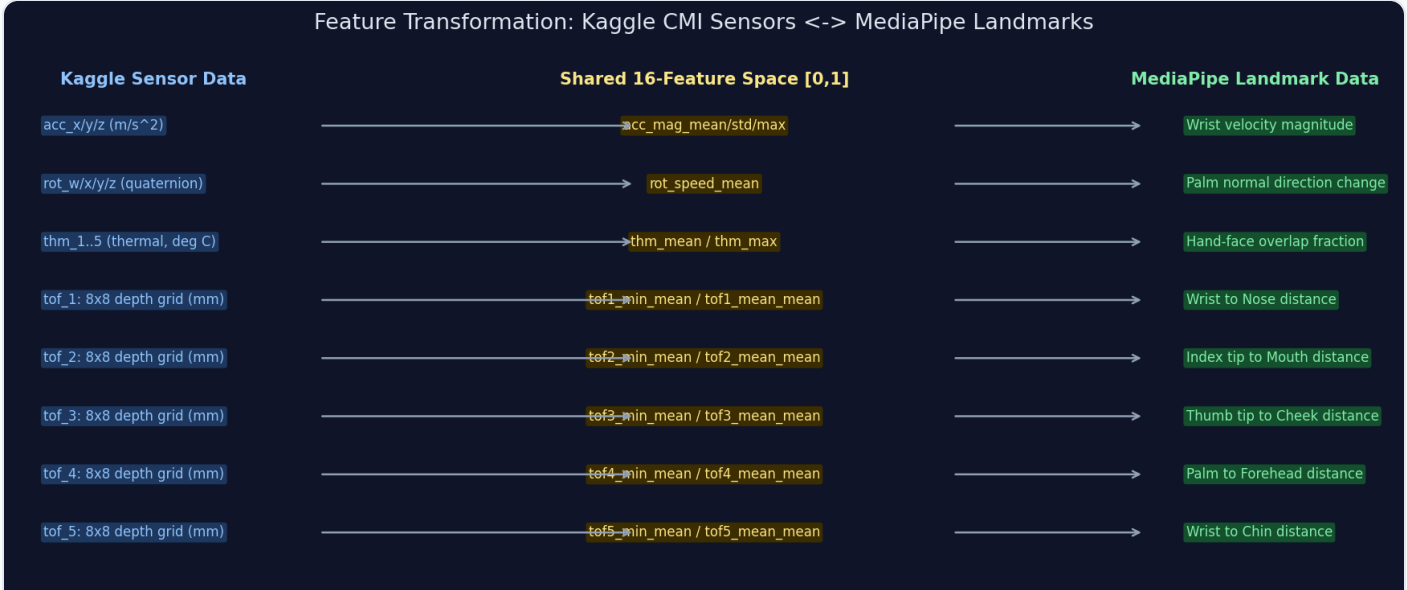


Figure 1: The proposed feature mapping from Kaggle sensor columns to MediaPipe landmark distances, through a shared 16-feature normalized space.

3.2 Feature Engineering (16 Features)

#	Feature Name	Kaggle Source	MediaPipe Analog
1	acc_mag_mean	Mean of sqrt(acc_x ² + acc_y ² + acc_z ²) over sequence	Mean wrist landmark velocity per frame
2	acc_mag_std	Std of acceleration magnitude	Std of wrist velocity
3	acc_mag_max	Max acceleration magnitude	Max wrist velocity
4	rot_speed_mean	Mean frame-to-frame quaternion change magnitude	Mean palm normal vector direction change
5	thm_mean	Mean of all 5 thermal sensors (°C)	Mean fraction of fingertips within 0.18 radius of nose
6	thm_max	Max thermal reading (°C)	Max hand-face overlap fraction per frame
7–8	tof1_min/mean_mean	Avg of per-row min/mean of ToF sensor 1 8×8 grid (mm)	Wrist-to-nose Euclidean distance (normalized coords)
9–10	tof2_min/mean_mean	ToF sensor 2	Index fingertip-to-mouth distance

11– 12	<code>tof3_min/mean_mean</code>	ToF sensor 3	Thumb tip-to-cheek distance
13– 14	<code>tof4_min/mean_mean</code>	ToF sensor 4	Palm center-to-forehead distance
15– 16	<code>tof5_min/mean_mean</code>	ToF sensor 5	Wrist-to-chin distance

3.3 Extraction Code (Kaggle Side)

```
def extract_features(group: pd.DataFrame) → dict:
    # 1-3: Accelerometer magnitude statistics
    acc_mag = np.sqrt(group["acc_x"]**2 + group["acc_y"]**2 + group["acc_z"]**2)
    feats["acc_mag_mean"] = acc_mag.mean()
    feats["acc_mag_std"] = acc_mag.std()
    feats["acc_mag_max"] = acc_mag.max()

    # 4: Quaternion frame-to-frame change = rotation speed
    rot = group[["rot_w", "rot_x", "rot_y", "rot_z"]].values
    diff = np.diff(rot, axis=0)
    feats["rot_speed_mean"] = np.sqrt((diff**2).sum(axis=1)).mean()

    # 5-6: Thermal sensors (skin contact / warmth)
    thm = group[[c for c in group.columns if c.startswith("thm_")]].values
    feats["thm_mean"] = thm.mean()
    feats["thm_max"] = thm.max()

    # 7-16: ToF 8x8 depth grids (per-row min + mean, averaged over sequence)
    for s in range(1, 6):
        cols = [f"tof_{s}_v{i}" for i in range(64)]
        mat = group[cols].values.astype(float)
        mat[mat < 0] = np.nan          # -1 = out of range
        row_mins = np.nanmin(mat, axis=1) # closest object per timestep
        row_means = np.nanmean(mat, axis=1) # avg proximity per timestep
        feats[f"tof{s}_min_mean"] = row_mins.mean()
        feats[f"tof{s}_mean_mean"] = row_means.mean()
```

3.4 Normalization

All 16 features were normalized to [0, 1] using `MinMaxScaler` fitted on the Kaggle training data. The MediaPipe features were manually mapped to [0, 1] by dividing by their expected maximum

values (velocity / 0.15, distances / 0.8, overlap fraction already in [0,1]).

4. Model Training Results

8,151

SEQUENCES

574,945

TOTAL ROWS

77%

TEST ACCURACY

0.837

AUC-ROC

0.82

BFRB F1

0.68

NO-BFRB F1

4.1 Classification Report (on Kaggle test set)

Class	Precision	Recall	F1-Score	Support
No-BFRB	0.66	0.69	0.68	576
BFRB	0.83	0.81	0.82	1,055
Accuracy			0.77	1,631

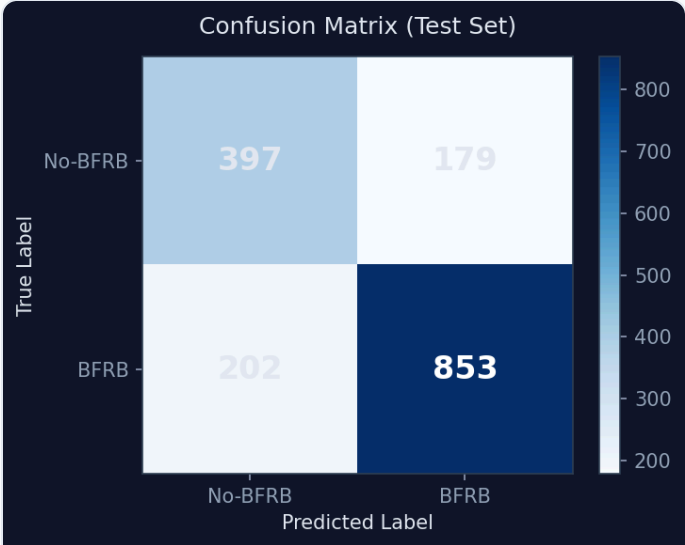


Figure 2: Confusion matrix on 1,631 test sequences. 853 BFRB correctly identified, but 179 No-BFRB sequences

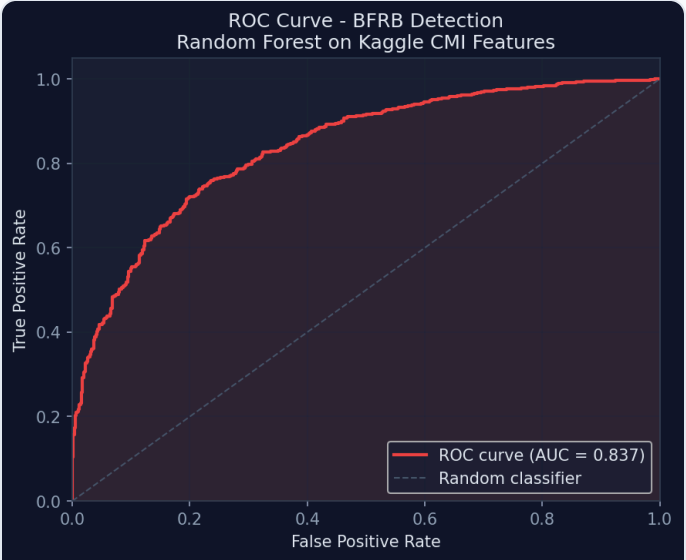


Figure 3: ROC curve showing AUC = 0.837. Solid performance on Kaggle data, but this does not transfer

misclassified as BFRB.

to webcam data.

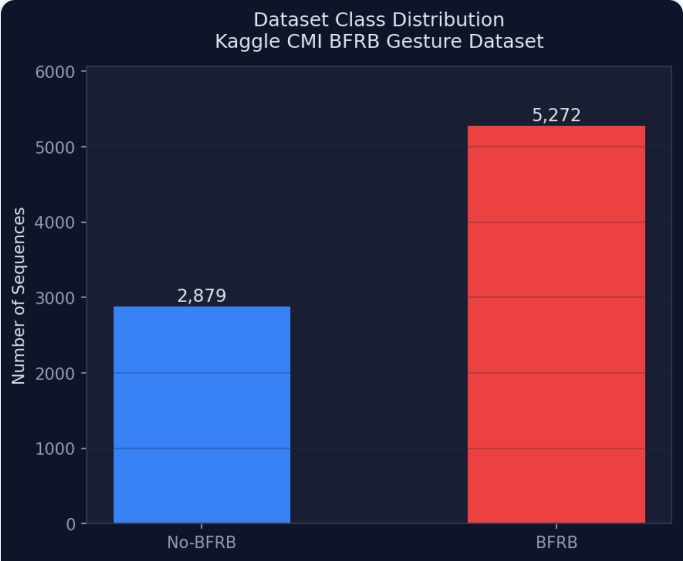


Figure 4: Dataset class distribution. BFRB sequences outnumber No-BFRB 5,272 to 2,879. `class_weight="balanced"` was used to compensate.

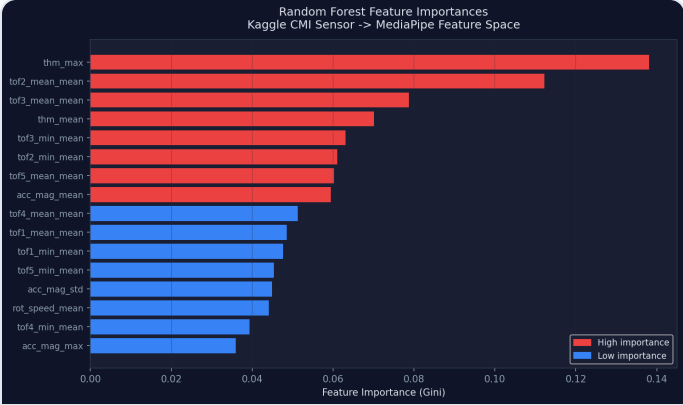


Figure 5: Random Forest feature importances. `thm_max` (thermal peak) was the most important feature at 13.8% — a critical issue for the domain gap.

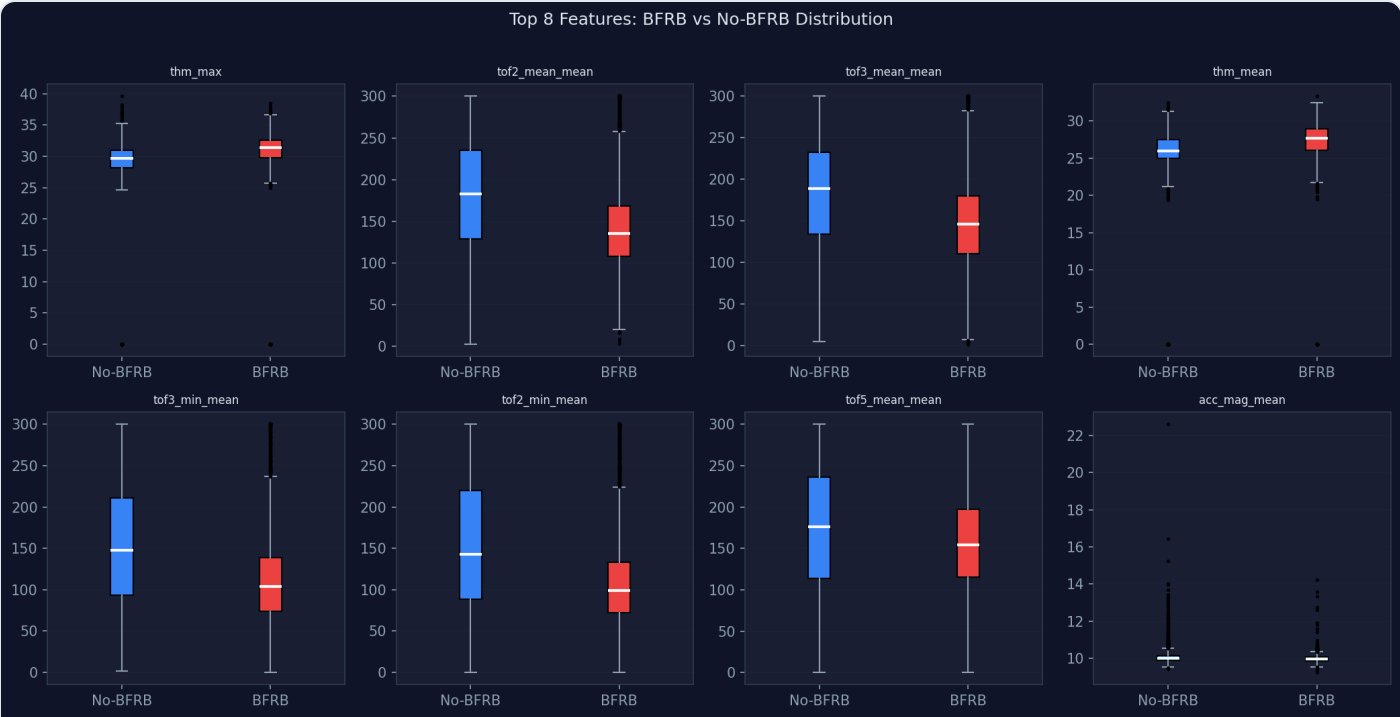


Figure 6: Box plots of the top 8 features by importance, split by class. Visible separation exists for `thm_max` and `tof2/3` features in Kaggle data, but this separation does not translate to MediaPipe-derived features.

5. Why This Approach Fails in Practice

ROOT CAUSE: DOMAIN GAP

The model was trained on physical wrist sensor readings and evaluated against normalized image-coordinate landmark distances. These are fundamentally different data types. The decision boundaries the Random Forest learned from Kaggle data do not generalize to MediaPipe's output space.

Problem 1: The Thermal-to-Overlap Mapping is Semantically Wrong

`thm_max` was the single most important feature (13.8% importance). The Kaggle thermal sensors measure actual skin temperature (~28–40°C) — they detect *sustained physical skin contact*, which is a true signal of hair pulling or skin picking. The MediaPipe analog I used was "fraction of fingertips within 0.18 normalized radius of the nose landmark."

This analog fires incorrectly because:

- Raising your hand anywhere in the camera frame can put fingertips within that radius
- The 0.18 threshold was not calibrated to any real-world measurement
- It activates for normal gestures like pointing at something above your head

Problem 2: Scale Mismatch After Normalization

Kaggle ToF readings range from ~50–300mm. After MinMaxScaler: a "touching face" reading (~60mm) normalizes to ~0.04. MediaPipe landmark distances range from 0.0 to ~0.8 (normalized image coordinates). A "touching face" landmark distance (~0.03) normalizes to ~0.04 too — so far so good. But the variance around those values is completely different:

- Kaggle "not touching": ~200mm → normalized ~0.63
- MediaPipe "hand at rest far from face": ~0.4 → normalized ~0.50

The RF's decision boundary was set at a threshold in Kaggle's normalized space. That threshold does not correspond to the same physical event in MediaPipe's space.

Problem 3: Wrist Sensor vs. Camera Perspective

The Kaggle device measures distance *outward from the wrist* in physical space. MediaPipe measures distances *in 2D normalized image coordinates*. A hand at the side of the frame vs. the center will have very different landmark distances to the face, even if physically the same proximity. Camera

angle, subject position, and frame cropping all affect MediaPipe distances in ways that have no analog in wrist sensor data.

Problem 4: No Temporal Modeling

The Kaggle features were computed per-sequence (aggregated statistics over an entire gesture). The MediaPipe system used a 30-frame rolling window (~1 second). This means:

- A 1-second window can capture mid-gesture, not a complete gesture like Kaggle sequences
- Transition frames (hand moving toward face) get classified the same as sustained BFRB
- No dwell time modeling — the model has no concept of "how long has the hand been there"

Problem 5: Model Never Saw Camera Data

The Random Forest was trained entirely on wrist sensor data. It has never seen a single frame of camera-derived landmark data. Transfer learning without fine-tuning on target domain data is expected to underperform. The 77% accuracy metric is only valid for the Kaggle test set; real-world webcam accuracy was not measured but observed to be poor.

6. Summary: What Worked vs. What Failed

Component	Status	Notes
Kaggle data loading & parsing	WORKED	574,945 rows, 341 columns processed correctly
ToF 8x8 grid summarization	WORKED	Min/mean per row, averaged across sequence
Random Forest on Kaggle features	WORKED	77% accuracy, AUC 0.837 on Kaggle test set
Accelerometer → velocity mapping	REASONABLE	Semantic similarity; scale different but pattern preserved
Quaternion → palm normal mapping	REASONABLE	Both measure rotation rate; similar semantic meaning
ToF proximity → landmark distance mapping	DOMAIN GAP	Different scales; normalization does not fully bridge

Thermal → face overlap mapping	FAILED	Semantically wrong; thermal = skin contact, overlap = spatial proximity
Model generalization to MediaPipe	FAILED	False positive rate too high for usable detection
No temporal/dwell modeling	MISSING	Critical for BFRB vs. incidental touch discrimination

Conclusion

The Kaggle CMI → MediaPipe transformation is a valid *research exploration* but is not a viable production approach for this web application. The domain gap between physical sensor measurements and image-coordinate landmark data is fundamentally too large to bridge with simple normalization. The model achieved solid performance on sensor data (AUC 0.837) but cannot generalize to camera data without retraining on actual webcam-derived labeled examples.

The correct approach — supported by published research and existing BFRB apps (BFRBye, Hands Off, StopBitingNails) — is a rule-based detector with adaptive thresholds, temporal dwell-time modeling, and velocity filtering applied *directly* to MediaPipe landmarks, with no cross-domain model transfer.

7. Archived Files

File	Description
transform_and_train.py	Full Kaggle feature extraction + RF training pipeline
model/bfrb_model.pkl	Trained Random Forest (200 trees, balanced)
model/scaler.pkl	MinMaxScaler fitted on Kaggle features
model/feature_meta.json	Feature names, descriptions, normalization bounds
templates/index_kaggle_v1.html	Original web app using Kaggle-trained model

[report_assets/*.png](#)

Training charts: ROC, confusion matrix, feature importances, etc.

ENEB453 Web-Based Application Development — Spring 2026 — Nishanth Tiglao — Report generated April 13, 2026